

Model Evaluation: Placement and Project Guide

Model evaluation answers two different questions: **does the model make useful decisions?** and **are its predicted probabilities trustworthy?** Choose a metric from the error cost and class distribution before tuning the model. Never select a model using the test set.

Shared setup used by the examples

```
import numpy as np
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

X, y = load_breast_cancer(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, stratify=y, random_state=42
)
model = make_pipeline(StandardScaler(), LogisticRegression(max_iter=2000))
model.fit(X_train, y_train)
score = model.predict_proba(X_test)[:, 1] # probability of positive class
pred = (score >= 0.5).astype(int)
```

`y_test`, `pred`, and `score` below mean true labels, hard predictions, and positive-class probabilities. For multiclass tasks, use a score per class and state whether averaging is `micro`, `macro`, or `weighted`.

Accuracy

1. Overview

Accuracy is the fraction of predictions that are correct. It is useful when classes are similarly frequent and false positives and false negatives have similar cost, such as balanced quality inspection.

2. Intuition

If a model gets 92 of 100 exam answers right, its accuracy is 92%. It treats every mistake identically.

3. Prerequisites

Binary/multiclass labels, train-validation-test splits, and the idea of a decision threshold.

4. Core Concepts

- **Correct prediction:** $y_{\text{hat}} == y$; it matters because accuracy counts only this outcome. Example: 8 correct of 10. Interview: accuracy ignores *which* class was missed.
- **Class imbalance:** a 99% "not fraud" predictor gets 99% accuracy on 1% fraud. Interview: explain why this can be useless.
- **Subset accuracy:** for multilabel data, all labels of a sample must match; it is much stricter than per-label accuracy.

5. Algorithm / Working Process

Make a hard class prediction for each example, compare it with the true label, count matches, and divide by the number of examples. Threshold probabilities before binary accuracy.

6. Mathematical Foundation

For n examples, $\text{Accuracy} = (1/n) \sum_i 1[y_i = y_{\text{hat}_i}]$. In binary classification it is $(TP + TN)/(TP + TN + FP + FN)$. It is not a training loss because the indicator is discontinuous; models usually train with log loss instead.

7. Practical Implementation

```
from sklearn.metrics import accuracy_score
print(f"accuracy={accuracy_score(y_test, pred):.3f}")
```

8. Code Explanation

`accuracy_score` compares aligned arrays element by element. Keep labels in the same encoding and use test data that played no role in threshold/model selection.

9. Training / Evaluation

Use a stratified split for classification. Tune models on validation/cross-validation, then report one final test accuracy with class counts. Check train versus validation accuracy for overfitting.

10. Complexity and Cost

Evaluation is $O(n)$ time and $O(1)$ extra memory if streamed. It needs no GPU and almost no compute.

11. Common Use Cases

Balanced image classification, digit recognition, intent classification with comparable class costs, and top-level model dashboards.

12. Common Mistakes

Using it alone for rare positives; reporting training accuracy; changing a threshold on the test set; averaging batch accuracies without weighting by batch size.

13. Edge Cases / Limitations

It hides minority-class failure, confidence, ranking quality, and error cost. A high score may coexist with zero recall for a critical class.

14. Variations

- **Balanced accuracy:** mean recall over classes; use for imbalance; placement-important.
- **Top-k accuracy:** correct if truth is among k choices; use for many classes.
- **Multilabel Hamming accuracy:** evaluates labels independently; use when exact-set accuracy is too harsh.

15. Related Topics

Precision and recall identify error type; the confusion matrix exposes its components; log loss evaluates probability quality rather than only the final class.

16. Interview Questions

1. **Formula?** Correct predictions divided by all predictions.
2. **Why poor for fraud?** A majority-class model can score highly while finding no fraud.
3. **Is it threshold-dependent?** Yes, for probabilistic classifiers.
4. **Can it train a neural net directly?** No; it is non-differentiable.
5. **Accuracy vs balanced accuracy?** Balanced accuracy weights classes equally through recall.
6. **Multiclass formula?** Same fraction, with any matching class counted correct.
7. **When is it enough?** Balanced classes and equal error cost.
8. **Can it show calibration?** No.
9. **Why stratify?** To preserve class proportion across splits.
10. **What report beside it?** Confusion matrix and per-class precision/recall/F1.

17. Practice Tasks

Compute accuracy manually with NumPy; compare a dummy majority classifier; change a threshold from 0.2 to 0.8 and plot accuracy; report per-class results on CIFAR-10.

18. Project Ideas

- **Defect sorter:** balanced manufactured-part images; PyTorch + accuracy/confusion matrix; demonstrates vision basics.
- **Intent router:** support intents with equal priority; scikit-learn; shows evaluation discipline.
- **Model leaderboard:** compare classifiers under cross-validation; Pandas/sklearn; resume-ready experiment reporting.

19. Quick Revision

Key idea: fraction right. Formula: $(TP+TN)/N$. Use when errors/classes are comparable. Trap: impressive accuracy on imbalanced data. One-liner: accuracy is simple correctness, not minority protection.

20. Final Cheat Sheet

Item	Notes
Definition	Fraction of exactly correct labels
Input/output	Labels and hard predictions -> scalar in $[0,1]$
Steps	Predict, compare, average matches
Hyperparameters	Model threshold affects it
Pros/cons	Intuitive/ignores error type and confidence
Best use	Balanced, equal-cost classification

Precision

1. Overview

Precision measures how often a predicted positive is actually positive. It is central when false alarms are expensive: spam auto-deletion, an alert that wakes an operator, or a recommendation shown to a user.

2. Intuition

Of 20 emails marked spam, 18 truly are spam: precision is 90%. It asks, "Can I trust a positive alert?"

3. Prerequisites

Positive/negative class definition, confusion matrix, thresholding, and class imbalance.

4. Core Concepts

- **False positive (FP):** predicted positive but actually negative; precision penalizes it. Example: legitimate mail in spam. Interview: specify the positive class first.
- **Threshold trade-off:** raising a threshold usually raises precision but lowers recall. Interview: threshold is a product decision, not automatically 0.5.
- **Averaging:** macro averages class-wise values equally; weighted weighs support; micro pools decisions. Interview: choose macro when minority classes matter equally.

5. Algorithm / Working Process

Select a positive class and threshold. Count true positives and every predicted positive. Divide true positives by predicted positives; decide a **zero_division** policy if none are predicted.

6. Mathematical Foundation

Precision = $TP / (TP + FP)$ = $P(Y=1 \mid \hat{Y}=1)$. It is a conditional probability estimated from predictions. It does not use **FN**, so it can be 1.0 when the model predicts only one very certain positive.

7. Practical Implementation

```
from sklearn.metrics import precision_score
print(precision_score(y_test, pred, zero_division=0))
```

8. Code Explanation

`zero_division=0` makes the no-positive-predictions case explicit rather than emitting an undefined-metric warning. For multiclass data use `average="macro"` or name the target with `pos_label`.

9. Training / Evaluation

Split stratified, tune threshold on validation data against an acceptable false-positive rate or minimum precision, then freeze it for test evaluation. Inspect confidence intervals if positives are few.

10. Complexity and Cost

One pass through predictions: $O(n)$ time, constant extra memory; CPU-only.

11. Common Use Cases

High-confidence search results, content moderation actions, medical confirmatory tests, fraud investigation queues, and alerting.

12. Common Mistakes

Calling the wrong label positive; hiding zero predicted positives; optimizing precision alone; using weighted precision without revealing minority support; selecting threshold on test data.

13. Edge Cases / Limitations

Precision changes with class prevalence and ignores missed positives. It cannot compare deployment environments with very different base rates without care.

14. Variations

- **Precision@k**: precision among k ranked items; recommender/search essential.
- **Macro precision**: equal class importance; placements.
- **Average precision**: threshold-free summary of a PR curve; ranking tasks.

15. Related Topics

Recall captures missed positives; F1 balances them; PR curves show precision over thresholds; positive predictive value is the medical name for precision.

16. Interview Questions

1. **Formula?** $TP / (TP + FP)$.
2. **What error hurts it?** False positives.
3. **Precision 1, recall low?** The model predicts few positives, all correct.

4. **Does prevalence matter?** Yes, strongly.
5. **Raise threshold effect?** Usually precision increases and recall decreases.
6. **Why zero division?** No predicted positive gives denominator zero.
7. **Precision vs specificity?** Precision conditions on prediction; specificity conditions on actual negatives.
8. **When prioritize it?** False alerts are costly.
9. **What is precision@k?** Correct items among first k results.
10. **Can a classifier maximize it trivially?** Yes, by predicting almost no positives.

17. Practice Tasks

Manually count TP/FP; tune a validation threshold for precision ≥ 0.95 ; compare macro and micro precision; implement precision@10 for a ranked list.

18. Project Ideas

- **Spam quarantine:** optimize precision for automatic blocking; sklearn + Enron/SpamAssassin; shows threshold design.
- **Safety alert triage:** rank toxic messages; transformers + Jigsaw data; demonstrates false-alarm control.
- **Product search audit:** compute precision@k; sentence embeddings + retail catalogue; recommender relevance project.

19. Quick Revision

Key idea: trust a positive. Formula: $\text{TP} / (\text{TP} + \text{FP})$. Use when false alarms hurt. Trap: predicting almost nothing. One-liner: precision asks whether positive predictions are correct.

20. Final Cheat Sheet

Item	Notes
Definition	Correct fraction of predicted positives
Input/output	Labels and hard predictions -> $[0, 1]$
Main steps	Count TP and FP
Hyperparameters	Threshold, class averaging
Pros/cons	Controls false alerts/ignores missed positives
Best use	High-confidence actions and ranking@k

Recall

1. Overview

Recall, also called sensitivity or true positive rate, measures how many actual positives a system finds. It matters when missing a positive is dangerous or costly, such as cancer screening or fraud detection.

2. Intuition

If 100 fraudulent transactions occurred and the model flags 85, recall is 85%. It asks, "How much of the real positive population did we catch?"

3. Prerequisites

Confusion matrix, positive-class choice, probability thresholds, and sampling bias.

4. Core Concepts

- **False negative (FN):** an actual positive missed by the model; recall penalizes it. Interview: describe its business cost.
- **Sensitivity:** another name for recall in diagnostics. Example: detecting diseased patients. Interview: distinguish it from specificity.
- **Recall@k:** fraction of relevant catalog items retrieved within k; useful in retrieval/RAG.

5. Algorithm / Working Process

After producing hard labels, count actual positives. Count those marked positive by the model. Divide the second count by the first.

6. Mathematical Foundation

$\text{Recall} = \text{TP} / (\text{TP} + \text{FN}) = P(\hat{Y}=1 \mid Y=1)$. With a binary score threshold, it is the y-axis of ROC space. Lowering the threshold generally increases it.

7. Practical Implementation

```
from sklearn.metrics import recall_score
print(recall_score(y_test, pred, zero_division=0))
```

8. Code Explanation

The function compares predicted positives with all true positives. In a multiclass report, use `average=None` first to see each class before selecting a summary average.

9. Training / Evaluation

Set a validation target, for example $\text{recall} \geq 0.98$, then select the highest-precision threshold meeting it. Ensure positives in test data represent the deployment population; otherwise recall will not transfer.

10. Complexity and Cost

$O(n)$ evaluation time and constant extra memory. Training cost is unchanged; class weighting/oversampling may alter training.

11. Common Use Cases

Disease screening, fraud candidate generation, intrusion detection, e-discovery, defect detection, RAG document retrieval.

12. Common Mistakes

Confusing recall with precision; reporting recall without number of positives; ignoring label noise; lowering threshold without measuring the resulting alert volume; using a random split for time-dependent fraud.

13. Edge Cases / Limitations

Always predicting positive gives recall 1 but can be operationally useless. Recall cannot show false-positive burden or probability calibration.

14. Variations

- **Macro recall / balanced accuracy:** treats classes equally; placement-important.
- **Recall@k:** retrieval/recommendation; important for RAG.
- **Recall at fixed precision:** operational constraint; strong project metric.

15. Related Topics

Precision measures alert quality; specificity measures true-negative detection; ROC uses recall/TPR, while PR is more informative for rare positives.

16. Interview Questions

1. **Formula?** $TP / (TP + FN)$.
2. **Other name?** Sensitivity or true positive rate.
3. **What error hurts it?** False negatives.
4. **What maximizes it trivially?** Predicting every sample positive.
5. **Threshold relationship?** Lower threshold usually increases recall.
6. **Recall vs specificity?** Recall is on actual positives; specificity on actual negatives.
7. **Why important in RAG?** Missing the answer-bearing document limits the generator.
8. **Does it use FP?** No.
9. **When prioritize?** Missing a positive has high cost.
10. **How choose threshold?** On validation data using a cost/recall requirement.

17. Practice Tasks

Calculate sensitivity by hand; tune for 95% recall; evaluate recall@5 in document retrieval; compare recall after class weighting.

18. Project Ideas

- **Fraud candidate finder:** maximize recall before manual review; XGBoost/sklearn + credit-card data; risk-focused story.
- **RAG retrieval benchmark:** report recall@k; FAISS + BEIR; directly relevant to LLM engineering.
- **Defect screening:** detect all likely faulty parts; torchvision + MVTec AD; demonstrates safety trade-offs.

19. Quick Revision

Key idea: find actual positives. Formula: $TP / (TP + FN)$. Use when misses hurt. Trap: 100% recall can mean every item is flagged. One-liner: recall measures coverage of the positive class.

20. Final Cheat Sheet

Item	Notes
Definition	Fraction of actual positives found
Input/output	Labels and predictions -> $[0, 1]$
Main steps	Count TP and FN
Hyperparameters	Threshold, class weights
Pros/cons	Protects against misses/ignores false alerts
Best use	Screening, detection, retrieval

F1-score

1. Overview

F1-score combines precision and recall into one number. It is a useful default for imbalanced classification when false positives and false negatives both matter and neither has a clearly larger cost.

2. Intuition

F1 is high only if both alert quality and positive coverage are high. It behaves like a harsh average: precision 1.0 and recall 0.1 produce F1 about 0.18, not 0.55.

3. Prerequisites

Precision, recall, harmonic mean, confusion matrices, and class averaging.

4. Core Concepts

- **Harmonic mean:** gives low values disproportionate influence; prevents one excellent component hiding one poor component. Interview: contrast with arithmetic mean.
- **F-beta:** weights recall more ($\beta > 1$) or precision more ($\beta < 1$). Example: F_2 for screening. Interview: β is chosen from error cost.
- **Averaging:** binary, micro, macro, and weighted definitions answer different multiclass questions.

5. Algorithm / Working Process

Compute TP, FP, FN at a chosen threshold, calculate precision and recall, then take their harmonic mean. Select threshold on validation data if F1 is the target.

6. Mathematical Foundation

$F1 = 2PR/(P+R) = 2TP/(2TP+FP+FN)$. $F_beta = (1+beta^2)PR/(beta^2 P+R)$. TN does not appear, which makes F1 useful when negatives dominate but means it ignores true-negative performance.

7. Practical Implementation

```
from sklearn.metrics import f1_score
print(f1_score(y_test, pred, average="binary", zero_division=0))
```

8. Code Explanation

`average="binary"` assumes labels are binary and 1 is positive. Use `average="macro"` for equal class importance; use `average="weighted"` only when support-weighting is intended.

9. Training / Evaluation

Cross-validate model parameters, then sweep validation thresholds to maximize F1 or meet an F_beta requirement. Report precision and recall beside F1 because two models can share F1 with different operational behavior.

10. Complexity and Cost

$O(n)$ after inference; no GPU requirement. A threshold sweep over m candidates is $O(n \log n)$ if scores are sorted.

11. Common Use Cases

Information extraction, spam detection, toxic-content detection, entity recognition, and imbalanced classification baselines.

12. Common Mistakes

Treating F1 as cost-aware by default; hiding precision/recall; using weighted F1 as proof minority performance is good; calculating from averaged P/R instead of pooled counts; tuning on test data.

13. Edge Cases / Limitations

F1 ignores TN, calibration, and ranking below the threshold. It is unsuitable if one error type is much more severe or if true negatives are valuable.

14. Variations

- **F2:** recall-focused; screening and retrieval.
- **F0.5:** precision-focused; auto-actions.
- **Macro F1:** equal weight per class; placement-essential for imbalanced multiclass tasks.

15. Related Topics

Precision and recall are its inputs; PR-AUC summarizes threshold behavior; MCC uses all four confusion-matrix cells and is often more balanced.

16. Interview Questions

1. **Formula?** $2PR / (P+R)$.
2. **Why harmonic mean?** A low P or R pulls it down strongly.
3. **Does F1 include TN?** No.
4. **When use F2?** Missing positives costs more.
5. **F1 of P=R=0.8?** 0.8.
6. **Why report P/R too?** Same F1 can hide different trade-offs.
7. **Macro vs micro F1?** Macro averages class scores; micro pools all decisions.
8. **Can all-positive predictions score well?** Sometimes, if prevalence is high.
9. **Threshold-dependent?** Yes.
10. **Main limitation?** It does not encode business cost or calibration.

17. Practice Tasks

Derive F1 from TP/FP/FN; compare F1 and F2 threshold choices; calculate macro/micro/weighted F1 for a 3-class matrix; find a model with same F1 but different P/R.

18. Project Ideas

- **Resume skill extractor:** optimize entity-level F1; spaCy/transformers + job postings; strong NLP project.
- **Toxic-comment filter:** compare F1 and F0.5; Hugging Face + Jigsaw; shows safety trade-offs.
- **Invoice anomaly detector:** tune F2 for missed anomalies; sklearn + synthetic/ERP data; business framing.

19. Quick Revision

Key idea: balance P and R. Formula: $2TP / (2TP+FP+FN)$. Use for imbalanced, equal-ish error cost. Trap: ignores TN and costs. One-liner: F1 refuses to reward a model that is good at only precision or recall.

20. Final Cheat Sheet

Item	Notes
Definition	Harmonic mean of precision and recall
Input/output	Labels and predictions -> $[0, 1]$
Main steps	Count TP/FP/FN, compute harmonic mean
Hyperparameters	Threshold, beta, averaging
Pros/cons	Compact imbalance metric/ignores TN and calibration
Best use	Detection and extraction

Confusion matrix

1. Overview

A confusion matrix is a table of actual versus predicted classes. It is the most diagnostic classification evaluation tool because nearly every common hard-label metric is derived from its cells.

2. Intuition

For a binary test it is a scorecard: correctly caught positives (TP), correctly rejected negatives (TN), false alarms (FP), and misses (FN). The error locations reveal what to fix.

3. Prerequisites

Class labels, positive-class definition, matrix indexing, and train/test separation.

4. Core Concepts

- **Rows/columns:** sklearn rows are true classes and columns predictions. Interview: always state the convention.
- **Binary cells:** TP, TN, FP, FN support P/R/F1/specificity. Example: $\begin{bmatrix} \text{TN} & \text{FP} \\ \text{FN} & \text{TP} \end{bmatrix}$.
- **Normalization:** by true row reveals recall; by prediction column reveals precision; by all samples reveals prevalence/error share.

5. Algorithm / Working Process

For every (actual, predicted) pair, increment the corresponding cell. In multiclass classification make a $C \times C$ table; diagonal cells are correct and off-diagonal cells are confusions.

6. Mathematical Foundation

$C_{ij} = \sum_k 1[y_k=i \text{ and } \hat{y}_k=j]$. Binary metrics follow: $TP=C_{11}$, $FP=C_{01}$, $FN=C_{10}$, $TN=C_{00}$ for the stated layout. Row-normalized $C_{ij}/\sum_j C_{ij}$ estimates $P(\text{pred}=j \mid \text{true}=i)$.

7. Practical Implementation

```
from sklearn.metrics import ConfusionMatrixDisplay, confusion_matrix
cm = confusion_matrix(y_test, pred, labels=[0, 1])
print(cm) # [[TN, FP], [FN, TP]]
ConfusionMatrixDisplay(cm, display_labels=["negative", "positive"]).plot()
```

8. Code Explanation

Passing `labels` fixes order even when a class is absent in a split. The plot makes systematic errors visible; add `normalize="true"` to `from_predictions` for class recall view.

9. Training / Evaluation

Build it only on held-out predictions after all tuning. Slice it by source, time, demographic group, or class to discover drift and fairness issues. Keep raw counts with percentages; percentages alone hide sample size.

10. Complexity and Cost

$O(n)$ updates and $O(C^2)$ storage. Large-label problems can make a dense matrix impractical; store sparse/error subsets instead.

11. Common Use Cases

Medical triage audits, defect-category debugging, OCR character errors, support-intent routing, and fairness/error analysis.

12. Common Mistakes

Reversing rows and columns; not naming positive class; showing only normalized values; ignoring rare classes; evaluating a threshold selected on test; treating multiclass one-vs-rest cells as independent.

13. Edge Cases / Limitations

It evaluates only hard decisions at one threshold and says nothing about probability confidence, rankings, or label ambiguity.

14. Variations

- **Normalized matrix:** compare class recalls/precisions across imbalance; placement-important.
- **Multilabel matrix:** one 2×2 matrix per label; use for tags.
- **Cost matrix:** multiply cells by business cost; use in deployment decisions.

15. Related Topics

Accuracy, P/R/F1, specificity, MCC, and kappa derive from it. ROC and PR instead vary the threshold over many such matrices.

16. Interview Questions

1. **What are its axes in sklearn?** Rows true, columns predicted.
2. **Binary layout?** $[[TN, FP], [FN, TP]]$ with labels $[0, 1]$.
3. **Diagonal means?** Correct predictions.
4. **Why normalize by row?** To inspect recall per actual class.
5. **What shows precision per prediction?** Column normalization.
6. **Can it evaluate probabilities?** Only after choosing a threshold.
7. **Multiclass size?** Number of classes squared.
8. **Why raw counts?** A 100% rate from one example is weak evidence.
9. **How find class confusions?** Inspect large off-diagonal cells.
10. **Which metrics derive from it?** Accuracy, precision, recall, F1, MCC, kappa.

17. Practice Tasks

Create a matrix manually; plot raw and row-normalized matrices; identify the most confused CIFAR-10 pair; add a dollar cost to FP/FN and compare thresholds.

18. Project Ideas

- **OCR error explorer:** clickable character confusion matrix; PyTorch + EMNIST; compelling visual debugging.
- **Support router audit:** inspect misrouted ticket intents; sklearn + CLINC150; practical operations focus.
- **Fairness monitor:** per-group matrices with confidence intervals; Pandas/sklearn + Adult dataset; responsible-AI value.

19. Quick Revision

Key idea: actual-versus-predicted count table. Formula: `C_ij=count(true=i,pred=j)`. Use to diagnose errors. Trap: swapped axes. One-liner: the confusion matrix is the source table behind classification metrics.

20. Final Cheat Sheet

Item	Notes
Definition	Counts for every true/predicted pair
Input/output	Labels, predictions -> <code>C</code> x <code>C</code> matrix
Main steps	Increment the matching cell
Hyperparameters	Label order, normalization
Pros/cons	Very interpretable/one threshold, grows with classes
Best use	Error diagnosis and metric derivation

ROC curve

1. Overview

The receiver operating characteristic (ROC) curve plots true positive rate against false positive rate across all score thresholds. It measures discrimination: whether positives tend to receive higher scores than negatives.

2. Intuition

Imagine moving a fraud-score cutoff from very strict to very lenient. Each cutoff catches more fraud (TPR) but also alarms on more legitimate transactions (FPR). ROC shows every trade-off.

3. Prerequisites

Probability scores, thresholds, recall/TPR, specificity, false-positive rate, and ranking.

4. Core Concepts

- **TPR:** recall $TP/(TP+FN)$, the vertical axis. Interview: it is sensitivity.
- **FPR:** $FP/(FP+TN)=1-\text{specificity}$, horizontal axis. Interview: it conditions on actual negatives, unlike precision.
- **Diagonal baseline:** random ranking has expected AUC 0.5. A curve nearer top-left is better.

5. Algorithm / Working Process

Sort unique model scores from high to low. Treat each as a threshold, make predicted positives above it, calculate TPR and FPR, then plot the resulting points including $(0,0)$ and $(1,1)$.

6. Mathematical Foundation

At threshold t , $TPR(t)=TP(t)/(TP(t)+FN(t))$ and $FPR(t)=FP(t)/(FP(t)+TN(t))$. ROC is $\{(FPR(t), TPR(t))\}$. AUC has the probabilistic interpretation $P(s(X+) > s(X-))$ plus half probability of a tie.

7. Practical Implementation

```
from sklearn.metrics import RocCurveDisplay
RocCurveDisplay.from_predictions(y_test, score)
```

8. Code Explanation

Pass continuous scores, not **pred**; hard labels generate only one meaningful operating point. The display calculates thresholds and plots FPR/TPR.

9. Training / Evaluation

Use validation ROC to select a threshold only when FPR/TPR reflects the operating constraint, such as TPR at $FPR \leq 1\%$. On imbalanced data also inspect a PR curve because ROC can look optimistic.

10. Complexity and Cost

Sorting scores costs $O(n \log n)$ time and $O(n)$ memory in typical implementations. This is evaluation-only and CPU-friendly.

11. Common Use Cases

Medical tests, fraud and intrusion detectors, binary model comparison, and threshold selection under false-positive limits.

12. Common Mistakes

Feeding hard labels; interpreting ROC as calibration; choosing a threshold by visual closeness to top-left without costs; relying on ROC alone for a 0.1% positive class; leaking test thresholds.

13. Edge Cases / Limitations

ROC is less revealing for extreme imbalance because many TNs make FPR appear tiny. It does not expose precision or decision prevalence.

14. Variations

- **One-vs-rest ROC:** multiclass curve per class; placement-important.
- **Partial AUC:** only FPR range that operations permit; production-relevant.
- **DET curve:** false-negative versus false-positive rates on transformed axes; biometrics/speech.

15. Related Topics

AUC is its scalar area; PR curves replace FPR with precision; calibration curves evaluate whether score values themselves are probabilities.

16. Interview Questions

1. **Axes?** FPR on x, TPR on y.
2. **What is TPR?** Recall.
3. **What is FPR?** $FP / (FP + TN)$.
4. **Random baseline?** Diagonal, AUC 0.5.
5. **Need probabilities?** Scores or probabilities, not hard labels.
6. **Threshold-dependent curve?** It summarizes all thresholds.
7. **Why weak under imbalance?** TN abundance can keep FPR deceptively small.
8. **Best point?** Depends on error costs/constraints.
9. **Does ROC assess calibration?** No.
10. **Multiclass method?** One-vs-rest or one-vs-one with stated averaging.

17. Practice Tasks

Plot ROC for logistic regression and random scores; calculate TPR/FPR at three thresholds; compare ROC and PR under simulated 1% positives; select maximum TPR subject to $FPR \leq 0.02$.

18. Project Ideas

- **Fraud threshold console:** choose threshold under FPR caps; Streamlit/sklearn + credit-card data; operations-ready evaluation.
- **Medical classifier comparison:** ROC plus CIs; PyTorch + breast-cancer images; research internship framing.
- **Drift ROC tracker:** compare monthly ROC curves; MLflow/Pandas; MLOps relevance.

19. Quick Revision

Key idea: all TPR/FPR trade-offs. Formula: $TPR = TP / (TP + FN)$, $FPR = FP / (FP + TN)$. Use for ranking/false-positive constraint. Trap: use scores, not labels. One-liner: ROC asks how well positives rank ahead of negatives.

20. Final Cheat Sheet

Item	Notes
Definition	TPR-versus-FPR curve across thresholds
Input/output	Binary labels and scores -> curve
Main steps	Sort scores, sweep threshold, count rates
Hyperparameters	Operating threshold selected afterward
Pros/cons	Threshold-independent discrimination/can flatter rare-positive tasks
Best use	Binary ranking and FPR-constrained decisions

AUC

1. Overview

Area under the ROC curve (ROC-AUC) compresses the ROC curve into one threshold-independent ranking statistic. It compares classifiers before a production threshold is fixed.

2. Intuition

Randomly pick one positive and one negative. AUC is the chance the model gives the positive a higher score. AUC 0.90 means this happens about 90% of the time.

3. Prerequisites

ROC curves, integration, probability scores, ties, and class imbalance caveats.

4. Core Concepts

- **Ranking statistic:** AUC depends on score order, not exact values. Interview: monotonic transformations preserve it.
- **Interpretation:** 0.5 random, 1 perfect, below 0.5 means reversed ranking can improve it. Interview: it is not probability correctness.
- **Partial AUC:** evaluates relevant FPR range; useful when only low false-alarm rates are feasible.

5. Algorithm / Working Process

Obtain one continuous score per sample, form ROC points over thresholds, and numerically integrate the area, normally trapezoidally. Libraries compute it directly from ranks/scores.

6. Mathematical Foundation

$AUC = \int_0^1 TPR(FPR) dFPR$. Equivalently, for n_+ positives and n_- negatives, AUC is the normalized Mann-Whitney rank statistic: pairwise wins plus 0.5 ties divided by $n_+ n_-$.

7. Practical Implementation

```
from sklearn.metrics import roc_auc_score
print(f"ROC-AUC={roc_auc_score(y_test, score):.3f}")
```

8. Code Explanation

The second argument must rank the positive class higher; `predict_proba(...)[ :, 1]` meets that convention. For multiclass, pass a score matrix and specify `multi_class="ovr"` or `"ovo"`.

9. Training / Evaluation

Choose hyperparameters by cross-validated AUC only if ranking is the objective. Report uncertainty (bootstrap/DeLong) for small datasets. Afterwards select threshold with cost, precision/recall, or FPR constraints.

10. Complexity and Cost

Typically $O(n \log n)$ due to sorting and $O(n)$ storage. Pairwise interpretation should not be implemented literally ($O(n+ n-)$) for large data.

11. Common Use Cases

Credit scoring, medical risk ranking, churn prioritization, ad ranking, and comparison of binary classifiers.

12. Common Mistakes

Using labels rather than scores; calling AUC an accuracy percentage; using it as a calibrated-risk measure; overlooking PR-AUC in rare-event work; comparing AUCs without uncertainty or paired samples.

13. Edge Cases / Limitations

It equally weights all FPR regions and cannot choose a threshold. Two models with equal AUC can differ greatly where the business operates; ROC-AUC can mask rare-positive precision.

14. Variations

- **ROC-AUC:** default binary ranking summary; placement-essential.
- **PR-AUC / average precision:** better rare-positive focus; important in detection.
- **Partial AUC:** constrained FPR; production/research relevant.

15. Related Topics

ROC is the curve AUC integrates; log loss/calibration error test probability values; concordance index is a related ranking measure for survival models.

16. Interview Questions

1. **Meaning of 0.8 AUC?** 80% chance a random positive outranks a random negative.
2. **Random AUC?** 0.5.

3. **Why not accuracy?** AUC uses score ordering across thresholds.
4. **Are values calibrated?** No; any monotonic transform preserves AUC.
5. **Why not pairwise code?** It is quadratic in class counts.
6. **Can AUC be below 0.5?** Yes; reversing scores yields above 0.5.
7. **Why PR-AUC for fraud?** It reflects positive prediction quality under imbalance.
8. **Does AUC choose threshold?** No.
9. **How multiclass?** OvR/OvO with stated averaging.
10. **What is partial AUC?** Area limited to an FPR range.

17. Practice Tasks

Show that `score` and `2*score+1` have equal AUC; calculate AUC for five ranked examples; bootstrap a 95% AUC interval; compare ROC-AUC with AP on 1% positives.

18. Project Ideas

- **Loan-risk ranker:** prioritize manual review by AUC; LightGBM/sklearn + LendingClub; finance-ready narrative.
- **Triage benchmark:** paired AUC confidence intervals; PyTorch + UCI data; research rigor.
- **Ranking monitor:** track ROC-AUC by customer segment; Evidently/Pandas; MLOps monitoring use.

19. Quick Revision

Key idea: probability a positive outranks a negative. Formula: area under ROC. Use to compare ranking before thresholding. Trap: it is not calibration. One-liner: AUC measures ordering, not decision quality at a chosen cutoff.

20. Final Cheat Sheet

Item	Notes
Definition	Area beneath ROC; pairwise ranking probability
Input/output	Binary labels and scores -> scalar <code>[0,1]</code>
Main steps	Sweep thresholds, integrate TPR/FPR
Hyperparameters	Multiclass strategy, optional FPR cap
Pros/cons	Threshold-free ranking/no operating point or calibration
Best use	Risk and priority ranking

PR curve

1. Overview

A precision-recall (PR) curve plots precision against recall over thresholds. It concentrates on positive predictions, making it especially informative when the positive class is rare.

2. Intuition

For a fraud model with 1% fraud, operators care whether the alerts they investigate are fraud (precision) while also wanting to find fraud (recall). PR shows the full trade-off without being dominated by true negatives.

3. Prerequisites

Precision, recall, thresholding, score ranking, prevalence, and average precision.

4. Core Concepts

- **Baseline prevalence:** a random model's expected precision is roughly positive prevalence. Interview: 0.10 AP can be excellent at 1% prevalence, poor at 50%.
- **Operating point:** choose recall/precision meeting capacity or safety constraints. Example: precision $\geq$ 90% for review queue.
- **Average precision (AP):** standard step-wise area summary of PR curve; do not confuse it with a simple trapezoidal area.

5. Algorithm / Working Process

Rank samples by descending score. Starting with the highest score, add predictions one at a time (or score group at a time), calculate cumulative precision and recall, then plot precision against recall.

6. Mathematical Foundation

At t , $P(t) = TP(t) / (TP(t) + FP(t))$, $R(t) = TP(t) / (TP(t) + FN(t))$. In sklearn, $AP = \sum_n (R_n - R_{n-1}) P_n$, a recall-weighted step integral. Random baseline precision equals $pi = P(Y=1)$.

7. Practical Implementation

```
from sklearn.metrics import PrecisionRecallDisplay, average_precision_score
PrecisionRecallDisplay.from_predictions(y_test, score)
print("AP:", average_precision_score(y_test, score))
```

8. Code Explanation

`from_predictions` uses scores to generate thresholds. `average_precision_score` gives the common scalar summary; label it AP rather than generic AUC to avoid implementation ambiguity.

9. Training / Evaluation

Use stratified validation and compare to prevalence. Pick the threshold from validation according to a precision floor, recall floor, reviewer capacity, or expected cost. Recompute baseline after deployment prevalence changes.

10. Complexity and Cost

Sorting is $O(n \log n)$ time and $O(n)$ memory. It is cheap compared with inference.

11. Common Use Cases

Fraud, rare disease, anomaly detection, information retrieval, object detection, RAG retrieval, and content moderation.

12. Common Mistakes

Comparing AP across datasets with very different prevalence; using hard labels; reading PR curve from left to right as a fixed threshold order; calling AP identical to trapezoidal PR-AUC; ignoring a realistic review budget.

13. Edge Cases / Limitations

PR curves are sensitive to prevalence and can be noisy when positives are few. They ignore true negatives and do not assess calibration.

14. Variations

- **AP:** standard scalar PR summary; placement-essential.
- **Precision@k / recall@k:** fixed review/retrieval budget; important in search/RAG.
- **Interpolated PR:** used in some benchmarks (for example object detection); research relevant.

15. Related Topics

Precision and recall supply axes; F1 picks a point on the curve; ROC substitutes FPR for precision and is less diagnostic under severe imbalance.

16. Interview Questions

1. **Axes?** Recall x-axis, precision y-axis.
2. **Random baseline?** Positive-class prevalence.
3. **Why useful for imbalance?** It focuses on positive predictions, not TNs.
4. **Need scores?** Yes, to sweep thresholds.
5. **AP formula idea?** Sum precision weighted by recall increments.
6. **Is AP prevalence-sensitive?** Yes.
7. **PR vs ROC?** PR better conveys alert quality for rare positives.
8. **Does it assess calibration?** No.
9. **How choose a point?** Business precision/recall/capacity constraint.
10. **Why can it be noisy?** Few positives create large recall jumps.

17. Practice Tasks

Plot PR on 1%, 10%, 50% prevalence simulations; calculate precision@100; tune maximum recall with precision $\geq .9$; compare AP against ROC-AUC for the same scores.

18. Project Ideas

- **RAG retriever evaluation:** Recall@k and precision@k against BEIR; FAISS + sentence-transformers; LLM-engineering relevant.

- **Fraud review queue:** threshold under analyst capacity; sklearn + credit-card data; business decision project.
- **Object detector report:** per-class AP; PyTorch + COCO subset; CV benchmark fluency.

19. Quick Revision

Key idea: positive-alert quality versus coverage. Formula: AP is precision weighted by recall increments. Use for rare positives. Trap: baseline is prevalence. One-liner: a PR curve is often the honest view of an imbalanced detector.

20. Final Cheat Sheet

Item	Notes
Definition	Precision-versus-recall curve over thresholds
Input/output	Labels and scores -> curve/AP
Main steps	Rank scores, cumulatively count TP/FP
Hyperparameters	Threshold, k or precision floor
Pros/cons	Excellent for rare positives/prevalence-sensitive
Best use	Detection, retrieval, review queues

Log loss

1. Overview

Log loss, also called cross-entropy or negative log-likelihood, measures the quality of predicted probabilities. It is the standard differentiable training objective for probabilistic classifiers and heavily penalizes confident wrong predictions.

2. Intuition

Predicting 0.99 for an event that does not occur is much worse than predicting 0.51. Log loss gives the first a large penalty because the model claimed near certainty and was wrong.

3. Prerequisites

Probabilities, logarithms, sigmoid/softmax, maximum likelihood, gradients, and calibration.

4. Core Concepts

- **Proper scoring rule:** expected log loss is minimized by predicting the true conditional probability. Interview: accuracy does not have this property.
- **Confidence penalty:** $-\log(p)$ grows without bound as true-class probability approaches zero. Interview: clipping avoids numerical $\log(0)$.

- **Cross-entropy:** empirical cross-entropy between labels and predictive distribution; standard neural-network loss.

5. Algorithm / Working Process

For each example obtain a valid probability distribution. Select the probability assigned to its true class, take negative log, then average. During training, backpropagate this loss to adjust model parameters.

6. Mathematical Foundation

Binary: $L = -(1/n) \sum [y_i \log p_i + (1-y_i) \log (1-p_i)]$. Multiclass with one-hot y_{ic} : $L = -(1/n) \sum_i \sum_c y_{ic} \log p_{ic}$. For logits z , use numerically stable sigmoid/softmax cross-entropy rather than computing probabilities then logs manually.

7. Practical Implementation

```
from sklearn.metrics import log_loss
print("log loss:", log_loss(y_test, score, labels=[0, 1]))

# PyTorch: loss = torch.nn.BCEWithLogitsLoss()(logits, targets.float())
```

8. Code Explanation

scikit-learn receives probabilities. PyTorch's `BCEWithLogitsLoss` receives raw logits and combines sigmoid plus loss stably; for multiclass use `CrossEntropyLoss` with logits and integer class indices.

9. Training / Evaluation

Fit preprocessing on train only, minimize training loss, tune regularization/architecture on validation log loss when probability quality matters, and report test log loss plus calibration. A widening train-validation gap signals overfit.

10. Complexity and Cost

Metric computation is $O(nc)$. In neural nets it costs about one output-layer pass and is negligible relative to the forward/backward network computation; GPU helps only because model training does.

11. Common Use Cases

Logistic regression, softmax classifiers, language modeling (token cross-entropy), click-through prediction, risk estimation, and probabilistic forecasting.

12. Common Mistakes

Passing logits where probabilities are expected; passing probabilities to `BCEWithLogitsLoss`; reversing classes; allowing zero probability without clipping; comparing values across different label spaces without context; equating low loss with best F1 threshold.

13. Edge Cases / Limitations

It is sensitive to mislabeled outliers and severely punishes overconfidence. Good log loss does not guarantee a business-optimal hard-label threshold; class weighting changes its probabilistic interpretation.

14. Variations

- **Binary cross-entropy:** binary/multilabel; placement-essential.
- **Categorical cross-entropy:** mutually exclusive multiclass; placement-essential.
- **Focal loss:** downweights easy examples; object detection/imbalance, project/research useful.

15. Related Topics

Calibration error checks reliability after training; Brier score is another proper probability score; perplexity is $\exp(\text{cross-entropy})$ in language models.

16. Interview Questions

1. **Other names?** Cross-entropy and negative log-likelihood.
2. **Binary formula?** $-\left[y \log p + (1-y) \log (1-p)\right]$ averaged.
3. **Why logarithm?** Strongly penalizes assigning tiny probability to truth and yields useful gradients.
4. **Why logits loss?** Numerical stability.
5. **Can accuracy train a net?** No, it lacks useful gradients.
6. **Perfect prediction loss?** Approaches zero.
7. **Confident wrong prediction?** Very large loss.
8. **Multiclass target?** One-hot in formula; usually integer index in PyTorch.
9. **Does low log loss mean calibrated?** It encourages it but does not prove perfect calibration.
10. **Relation to maximum likelihood?** Minimizing it maximizes likelihood of observed labels.

17. Practice Tasks

Compute losses for correct $p=.6$ and wrong $p=.99$; implement stable sigmoid BCE; compare log loss and accuracy after temperature scaling; train a classifier with focal versus BCE loss.

18. Project Ideas

- **Churn probability estimator:** calibrate and minimize log loss; sklearn + Telco Churn; business-ready probabilities.
- **Next-token mini-LM:** report cross-entropy/perplexity; PyTorch + Tiny Shakespeare; core DL skill.
- **Overconfidence audit:** identify high-loss errors; transformers + text data; shows model reliability work.

19. Quick Revision

Key idea: penalize incorrect confidence. Formula: negative log true-class probability. Use for probabilistic training/evaluation. Trap: logits versus probabilities API mismatch. One-liner: log loss rewards honest probabilities and punishes confident mistakes.

20. Final Cheat Sheet

Item	Notes
Definition	Mean negative log probability assigned to truth
Input/output	Labels and probabilities/logits -> nonnegative loss
Main steps	Select true-class probability, negative log, average
Hyperparameters	Class weights, label smoothing, focal gamma
Pros/cons	Differentiable/proper/sensitive to noisy confident errors
Best use	Training and probability quality

Top-k accuracy

1. Overview

Top-k accuracy counts a prediction as correct when the true class appears among the model's k highest-scoring classes. It is useful where many labels are plausible and showing multiple candidates is acceptable, such as ImageNet recognition or retrieval-assisted routing.

2. Intuition

If an image is a husky and the classifier's first three guesses include husky, top-3 is correct even if its top-1 guess is malamute. It evaluates candidate-list usefulness.

3. Prerequisites

Multiclass logits/probabilities, sorting/ranking, top-1 accuracy, and label indexing.

4. Core Concepts

- **Top-1 vs top-k:** top-1 is ordinary accuracy; larger k is less strict. Interview: top-k must be paired with intended UI/action.
- **Score order:** probabilities and logits give identical top-k ranking under softmax. Interview: no softmax is needed just to rank logits.
- **Candidate set:** the true label needs only membership, not first rank; it does not assess calibration.

5. Algorithm / Working Process

For each example, sort class scores or use partial selection to find k largest indices. Mark success if the true label is in that set. Average successes over all examples.

6. Mathematical Foundation

$\text{TopKAcc} = (1/n) \sum_i 1[y_i \text{ in TopK}(s_i)]$. It is non-decreasing in k; with $k=C$ it is always 1 (assuming label is in class vocabulary), so k must be meaningful.

7. Practical Implementation

```
from sklearn.metrics import top_k_accuracy_score
proba = model.predict_proba(X_test)
print(top_k_accuracy_score(y_test, proba, k=2, labels=model.classes_))
```

8. Code Explanation

`proba` has shape `(n_samples, n_classes)` and columns must align with `labels`. State `k` in reports; a top-5 score cannot be compared directly with top-1.

9. Training / Evaluation

Keep a representative test taxonomy. Track top-1 and top-k because a high top-k can hide poor autonomous decisions. Improve with more data, label cleanup, hierarchical labels, or a reranker—not by increasing `k` without capacity rationale.

10. Complexity and Cost

Full sorting per sample is $O(C \log C)$; selecting `k` items can be $O(C)$ with partial selection. Memory is $O(C)$ per sample scores, often the output-layer bottleneck for huge vocabularies.

11. Common Use Cases

Image classification, large-vocabulary speech, product category suggestions, entity linking, and multi-candidate intent routing.

12. Common Mistakes

Using it for binary decisions; not aligning score columns and labels; reporting top-5 without top-1; choosing `k` after viewing test outcomes; confusing ranking success with calibrated probability.

13. Edge Cases / Limitations

Large `k` can inflate apparent quality and may exceed user/compute capacity. It says no rank within `k` and no confidence information; ambiguous/duplicate labels complicate ground truth.

14. Variations

- **Top-1:** standard accuracy; placement-essential.
- **Recall@k:** same membership idea in retrieval; crucial for RAG.
- **Mean reciprocal rank (MRR):** rewards placing truth earlier; search/recommendation useful.

15. Related Topics

Accuracy is top-1; ranking metrics such as MRR/NDCG care about ordering; cross-entropy optimizes the score distribution used for top-k.

16. Interview Questions

1. **Definition?** Truth appears among k highest-scoring classes.
2. **Top-1 equals?** Ordinary multiclass accuracy.
3. **Need softmax?** No, logits preserve softmax ranking.
4. **Why use it?** Multiple suggestions can be useful.
5. **k=C score?** Always 1.
6. **Does it assess calibration?** No.
7. **What must align?** Score columns and class labels.
8. **Complexity naive?** $O(C \log C)$ per example.
9. **Top-k vs MRR?** Top-k ignores rank inside the list.
10. **Binary use?** Usually not useful beyond top-1.

17. Practice Tasks

Implement top-k with `np.argmaxpartition`; verify top-1 equals accuracy; evaluate k=1,3,5 on CIFAR-10; find examples where top-5 hides poor first choice.

18. Project Ideas

- **Plant-disease assistant:** show top-3 diagnoses; torchvision + PlantVillage; user-facing CV value.
- **Product taxonomy suggester:** top-5 category candidates; transformers + retail data; practical workflow automation.
- **Entity linker benchmark:** compare top-k and MRR; sentence-transformers + Wikipedia entities; NLP retrieval skill.

19. Quick Revision

Key idea: truth is in candidate list. Formula: $\mathbb{1}[y \text{ in TopK}]/n$. Use for many-class suggestions. Trap: k can make results look inflated. One-liner: top-k measures candidate coverage, not first-choice certainty.

20. Final Cheat Sheet

Item	Notes
Definition	Correct if true class is among k largest scores
Input/output	True labels + class score matrix -> <code>[0,1]</code>
Main steps	Select top k indices, test membership
Hyperparameters	k
Pros/cons	Matches suggestion workflows/ignores rank and calibration inside k
Best use	Large-label classification and candidate Uls

Matthews correlation coefficient

1. Overview

Matthews correlation coefficient (MCC) is a single balanced binary classification metric built from all TP, TN, FP, and FN. It is particularly valuable for imbalanced data because it rewards correct classification of both classes.

2. Intuition

Think of binary labels and predictions as two yes/no variables. MCC is their correlation: +1 means perfect agreement, 0 means no better association than random, and -1 means complete inverse prediction.

3. Prerequisites

Confusion matrix, correlation intuition, class imbalance, and binary/multiclass labels.

4. Core Concepts

- **All four cells:** unlike F1, MCC includes TN; it matters when negatives are meaningful. Interview: quote this distinction.
- **Range:** $[-1, 1]$; 0 denotes no correlation, not necessarily 50% accuracy. Interview: prevalence changes accuracy baseline.
- **Degenerate predictions:** denominator can be zero when a class is absent in predictions/truth; implementations conventionally return 0.

5. Algorithm / Working Process

Create a confusion matrix, place its four binary counts in the MCC formula, and interpret sign and magnitude. For multiclass, use the generalized confusion-matrix form via a library.

6. Mathematical Foundation

$MCC = (TP * TN - FP * FN) / \sqrt{((TP + FP)(TP + FN)(TN + FP)(TN + FN))}$. Numerator rewards agreement beyond crossed errors; denominator normalizes marginals. It is equivalent to Pearson correlation for binary encodings in the nondegenerate case.

7. Practical Implementation

```
from sklearn.metrics import matthews_corrcoef
print("MCC:", matthews_corrcoef(y_test, pred))
```

8. Code Explanation

The function works for binary and multiclass labels, avoiding error-prone manual denominators. Still inspect confusion matrix to explain the coefficient to stakeholders.

9. Training / Evaluation

For imbalanced classification, choose model/threshold by cross-validated MCC if balanced performance is the desired objective. Report P/R and counts too, especially when product costs are asymmetric.

10. Complexity and Cost

$O(n)$ to construct counts; $O(C^2)$ storage if using a multiclass matrix. Negligible CPU cost.

11. Common Use Cases

Rare disease screening, bioinformatics, fraud, anomaly detection, and imbalanced binary benchmark comparison.

12. Common Mistakes

Calling MCC an accuracy; ignoring its negative range; selecting it when FP/FN costs are intentionally asymmetric; manually dividing by zero; comparing estimates from tiny minority samples without intervals.

13. Edge Cases / Limitations

MCC is less intuitive for nontechnical stakeholders and does not assess calibration or ranking. It cannot represent a bespoke cost matrix; one number can hide which class fails.

14. Variations

- **Binary MCC:** common use; placement-important.
- **Multiclass MCC:** generalized correlation from the matrix; useful for balanced multi-class reporting.
- **Markedness/informedness:** related diagnostic measures; research/interview extension.

15. Related Topics

F1 excludes TN while MCC includes it; Cohen's kappa is chance-corrected agreement; balanced accuracy averages recalls but is not correlation-based.

16. Interview Questions

1. **Range?** -1 to 1.
2. **Perfect/random/inverse?** +1, 0, -1.
3. **Formula inputs?** TP, TN, FP, FN.
4. **Why good for imbalance?** It uses all four cells and normalizes marginals.
5. **MCC vs F1?** MCC includes TN; F1 does not.
6. **Is 0 equal 50% accuracy?** No.
7. **Can it be multiclass?** Yes, generalized implementations exist.
8. **What if denominator zero?** Metric is degenerate; sklearn returns 0.
9. **Does it use probabilities?** No, hard predictions at a threshold.
10. **Main drawback?** Harder to communicate and not cost/calibration aware.

17. Practice Tasks

Calculate MCC for a skewed matrix; compare MCC/F1/accuracy for all-negative predictions; threshold-sweep maximum MCC; verify multiclass sklearn output.

18. Project Ideas

- **Rare-disease benchmark:** select threshold by MCC; sklearn + UCI datasets; strong metric choice story.
- **Protein-function classifier:** imbalanced labels; PyTorch + protein features; research-oriented framing.
- **Fraud model card:** explain MCC with matrix; Pandas/sklearn + credit-card data; responsible reporting.

19. Quick Revision

Key idea: balanced correlation of true and predicted labels. Formula: normalized $TP \cdot TN - FP \cdot FN$. Use for imbalanced hard-label classification. Trap: no calibration/cost story. One-liner: MCC is a robust one-number summary when both classes matter.

20. Final Cheat Sheet

Item	Notes
Definition	Correlation-like score from all confusion cells
Input/output	Labels and hard predictions -> $[-1, 1]$
Main steps	Count matrix, normalize agreement expression
Hyperparameters	Threshold
Pros/cons	Strong under imbalance/less intuitive, no probability quality
Best use	Rare-event balanced evaluation

Cohen's kappa

1. Overview

Cohen's kappa measures agreement between two categorical labelers after subtracting agreement expected from their marginal label frequencies. In ML it compares model predictions with ground truth; it is also vital for measuring agreement between human annotators.

2. Intuition

If two raters both mostly choose "normal," raw agreement can be high by chance. Kappa asks how much more they agree than would be expected if each kept their own label distribution but paired labels randomly.

3. Prerequisites

Confusion matrices, observed proportions, marginal probabilities, categorical annotation, and chance agreement.

4. Core Concepts

- **Observed agreement (p_o):** diagonal count divided by N; raw accuracy. Interview: kappa begins here.
- **Expected agreement (p_e):** sum of products of each rater's class marginals. Interview: it is not an independent data-generating model.

- **Weights:** quadratic/linear weighted kappa gives partial credit for close ordinal ratings. Example: severity 3 predicted as 2 is less wrong than 0.

5. Algorithm / Working Process

Build the two-rater confusion matrix. Compute observed diagonal proportion. Compute expected diagonal agreement from row and column marginals, then normalize the excess agreement by the maximum possible excess.

6. Mathematical Foundation

$\text{kappa} = (\text{p}_o - \text{p}_e) / (1 - \text{p}_e)$, where $\text{p}_o = \sum_i C_{ii} / N$ and $\text{p}_e = \sum_i (\text{row}_i / N)(\text{col}_i / N)$. Kappa is 1 for complete agreement, 0 for expected-by-marginals agreement, and can be negative for worse-than-chance agreement.

7. Practical Implementation

```
from sklearn.metrics import cohen_kappa_score
print(cohen_kappa_score(y_test, pred))
# For ordered ratings: cohen_kappa_score(y_test, pred, weights="quadratic")
```

8. Code Explanation

Inputs are two aligned categorical label sequences. `weights="quadratic"` is appropriate only when label distances are meaningful and ordered; do not use it for unordered classes like cat/dog/car.

9. Training / Evaluation

Use kappa to audit annotation quality before training: low inter-annotator kappa places a ceiling on supervised learning. For model selection, use it only when chance-corrected agreement aligns with the task; inspect per-class errors too.

10. Complexity and Cost

$O(n+C^2)$ time to count and calculate marginals, $O(C^2)$ matrix space. Cheap on CPU.

11. Common Use Cases

Medical image grading, sentiment/quality ratings, educational scoring, content-moderation annotation quality, and ordinal severity prediction.

12. Common Mistakes

Treating 0 as no raw agreement; using weighted kappa for nominal labels; quoting arbitrary interpretation bands as universal truth; ignoring prevalence paradox; comparing kappas across different label distributions.

13. Edge Cases / Limitations

Kappa is sensitive to prevalence and rater bias; high raw agreement can yield low kappa when one class dominates. It is not a replacement for class-wise error inspection or calibration metrics.

14. Variations

- **Unweighted kappa:** nominal labels; placement-important.
- **Linear/quadratic weighted kappa:** ordinal labels; important for rating tasks.
- **Fleiss' kappa:** more than two annotators; research/data-labeling relevant.

15. Related Topics

Accuracy is raw observed agreement; MCC is correlation-based and often used for binary classification; Krippendorff's alpha handles more annotators and missing data.

16. Interview Questions

1. **Formula?** $(p_o - p_e) / (1 - p_e)$.
2. **What does p_e mean?** Agreement expected from the two marginal label frequencies.
3. **Kappa 1/0/negative?** Perfect/chance-level/worse-than-chance agreement.
4. **Why not accuracy?** It corrects a form of chance agreement.
5. **When weighted?** Ordered classes with meaningful distances.
6. **Why not weighted for nominal labels?** Their distances have no meaning.
7. **What is prevalence paradox?** High raw agreement can coexist with low kappa under skewed marginals.
8. **Two annotators only?** Cohen's kappa; Fleiss' for many.
9. **Does it assess probabilities?** No.
10. **ML data-quality use?** Measure labeler consistency before trusting labels.

17. Practice Tasks

Compute p_o and p_e manually; compare accuracy/kappa on skewed labels; calculate quadratic kappa for ordinal ratings; measure annotator agreement before model training.

18. Project Ideas

- **Radiology label audit:** compare annotator and model kappa; sklearn + public chest X-ray labels; healthcare-data quality narrative.
- **Essay-grade predictor:** optimize quadratic kappa; transformers + ASAP dataset; ranking/ordinal evaluation skill.
- **Moderation guideline dashboard:** report inter-rater kappa by policy version; Pandas; governance/MLOps value.

19. Quick Revision

Key idea: agreement beyond marginal chance. Formula: $(p_o - p_e) / (1 - p_e)$. Use for labeler agreement and ordinal ratings. Trap: prevalence paradox. One-liner: kappa tells whether agreement is more than "both mostly picked the common label."

20. Final Cheat Sheet

Item	Notes
Definition	Chance-corrected categorical agreement
Input/output	Two aligned label sequences -> [-1,1]
Main steps	Build matrix, compute observed/expected agreement
Hyperparameters	Nominal vs linear/quadratic weights
Pros/cons	Useful for annotators/prevalence and bias sensitive
Best use	Data labeling and ordinal prediction

Calibration error

1. Overview

Calibration error measures whether predicted probabilities match observed frequencies. A calibrated model that gives 0.70 to a group of cases should see about 70% positives in that group. It matters whenever probabilities drive decisions, prices, risk, or human trust.

2. Intuition

Weather forecasts are calibrated if days forecast at 30% rain actually rain on about 30% of those days. A classifier can have high accuracy/AUC but be overconfident and therefore unsafe for probability-based decisions.

3. Prerequisites

Predicted probabilities, Bernoulli frequency, bins, reliability diagrams, log loss/Brier score, and validation splits.

4. Core Concepts

- **Reliability diagram:** bin predictions and compare mean confidence to empirical accuracy. Interview: points on diagonal are calibrated.
- **ECE:** weighted average absolute gap across bins; easy to report but depends on binning. Interview: ECE can hide within-bin miscalibration.
- **MCE:** maximum bin gap; catches worst bin but is noisy. Interview: report bin counts.
- **Post-hoc calibration:** fit Platt scaling/isotonic regression on a separate calibration set, never on final test labels.

5. Algorithm / Working Process

Collect held-out probabilities and labels. Partition probability range into bins. For each bin calculate average predicted confidence and observed positive frequency. Take a weighted difference summary; plot the pairs. Optionally learn a calibrator on validation data and test it on untouched data.

6. Mathematical Foundation

For bins B_m , $\text{conf}(B_m) = \text{mean}(p_i)$, $\text{acc}(B_m) = \text{mean}(y_i)$, and $n_m = |B_m|$: $\text{ECE} = \sum_m (n_m/n) |\text{acc}(B_m) - \text{conf}(B_m)|$. $\text{MCE} = \max_m |\text{acc}(B_m) - \text{conf}(B_m)|$. Perfect calibration means $P(Y=1 | \hat{P}_{\text{hat}}=p) = p$ for all p (where defined).

7. Practical Implementation

```
from sklearn.calibration import calibration_curve

prob_true, prob_pred = calibration_curve(y_test, score, n_bins=10,
strategy="quantile")
ece = np.mean(np.abs(prob_true - prob_pred)) # unweighted diagnostic only
print(list(zip(prob_pred.round(3), prob_true.round(3))), "rough ECE:", ece)
```

8. Code Explanation

`calibration_curve` returns per-bin observed frequency and mean prediction. The simple mean above is a quick visual diagnostic, not canonical weighted ECE because sklearn does not return bin counts; for a production ECE, implement/retain counts and weight gaps by n_m/n .

9. Training / Evaluation

Reserve calibration data or use cross-validated calibration. Evaluate calibration with reliability plot, log loss, Brier score, and ECE on untouched test data. Temperature scaling is common for deep nets; retest after data drift because calibration often changes before accuracy.

10. Complexity and Cost

Bin-based ECE is $O(n)$ time and $O(M)$ memory for M bins. Isotonic calibration needs sorted validation scores; temperature/Platt scaling is very cheap compared with model training.

11. Common Use Cases

Medical risk scores, loan default probabilities, demand forecasts, selective classification, ensembles, LLM confidence/routing, and human-in-the-loop decisions.

12. Common Mistakes

Calibrating/test-tuning on the same test set; treating confidence as calibration; using too few samples per bin; reporting ECE without bins/strategy; assuming AUC improvement implies calibration improvement; applying isotonic regression with too little calibration data.

13. Edge Cases / Limitations

ECE changes with bin count/placement and can estimate poorly in sparse bins. Marginal calibration can conceal subgroup miscalibration. Calibrating under one prevalence may fail after dataset shift.

14. Variations

- **ECE:** weighted binned absolute gap; placement-important.
- **Adaptive/quantile ECE:** equalizes bin sample counts; useful for skewed score distributions.
- **Classwise ECE:** per class in multiclass settings; research/production useful.
- **Brier score:** squared probability error, bin-free proper score; strong companion metric.

15. Related Topics

Log loss also rewards honest probabilities; ROC-AUC evaluates ranking only; Platt scaling, isotonic regression, and temperature scaling are calibration methods; conformal prediction targets coverage rather than calibration alone.

16. Interview Questions

1. **What is calibration?** Predicted probability equals observed frequency at that confidence.
2. **Can high AUC be miscalibrated?** Yes; AUC only cares about ranks.
3. **ECE formula idea?** Weighted average bin gap between accuracy and confidence.
4. **Why bins are a limitation?** Value changes with binning and sparse bins are noisy.
5. **Reliability diagram diagonal?** Perfect calibration.
6. **Overconfident model?** Confidence exceeds observed accuracy.
7. **Post-hoc methods?** Platt scaling, isotonic regression, temperature scaling.
8. **Why separate calibration set?** To avoid fitting/evaluating on the same labels.
9. **ECE vs log loss?** ECE is binned calibration diagnostic; log loss is a proper scoring loss.
10. **Why monitor after deployment?** Dataset/prevalence shift can break calibration.

17. Practice Tasks

Build weighted ECE with `np.digitize`; plot reliability before/after temperature scaling; compare equal-width and quantile bins; audit ECE by demographic subgroup.

18. Project Ideas

- **Credit-risk calibration service:** compare Platt/isotonic calibration; sklearn + LendingClub; probability-governance value.
- **Neural-net confidence audit:** temperature-scale image classifier; PyTorch + CIFAR-10; deep-learning reliability project.
- **LLM routing simulator:** send low-confidence requests to a stronger model; API logs + calibration dashboard; AI-engineering relevance.

19. Quick Revision

Key idea: 70% confidence should be right 70% of the time. Formula: weighted bin gap for ECE. Use when probabilities inform decisions. Trap: ECE depends on bins and test leakage. One-liner: calibration is probability honesty, not classification accuracy.

20. Final Cheat Sheet

Item	Notes
Definition	Agreement between confidence and observed frequency

Item	Notes
Input/output	Held-out probabilities and labels -> plot/ECE
Main steps	Bin scores, compare mean confidence and outcomes
Hyperparameters	Bin count/strategy, calibration method
Pros/cons	Makes risks actionable/bin-sensitive and data-hungry
Best use	Risk, decisions, uncertainty-aware systems

Metric Selection: a placement-ready summary

Use the error cost first, then validate on realistic data.

Situation	Primary metric	Report alongside
Balanced classes, equal errors	Accuracy	Confusion matrix
Costly false positives	Precision / precision@k	Recall, alert volume
Costly false negatives	Recall / recall@k	Precision, workload
Both error types matter under imbalance	F1 or MCC	Precision, recall, confusion matrix
Compare score rankings	ROC-AUC	PR-AUC for rare positives
Rare positive detector	AP / PR curve	Recall at precision/capacity
Probabilities drive actions	Log loss + calibration	Reliability diagram, Brier score
Many plausible classes	Top-k accuracy	Top-1, MRR if rank matters
Annotation/ordinal agreement	Cohen's kappa	Raw agreement and per-class matrix

Interview closing line: no metric is universally best; define the positive class, quantify FP/FN cost, use validation for choices, and reserve the test set for one final unbiased estimate.